

Library Management System

Documentation — All Functions, Concise

Data Structure: Python List | Algorithms: Merge Sort & Binary Search

Overview

Manages a book collection using **one list** (`self.books`) and **two algorithms** (Merge Sort, Binary Search). All 9 operations work directly on that list.

#	Function	Algorithm	Time	What it does
1	<code>display_all</code>	—	$O(n)$	Print every book numbered
2	<code>add_book</code>	—	$O(1)$	Append to end of list
3	<code>delete_book</code>	Linear scan	$O(n)$	Find by title, remove
4	<code>search_title_author</code>	Linear scan	$O(n)$	Substring match on title or author
5	<code>search_genre</code>	Linear scan	$O(n)$	Exact match on genre
6	<code>sort_by_year</code>	Merge Sort	$O(n \log n)$	Sort by year, asc or desc
7	<code>search_by_year</code>	Binary Search	$O(\log n)$	Find by year (sort asc first)
8	<code>lost_book_fine</code>	Linear scan	$O(n)$	price + days x \$0.50
9	<code>sort_by_author</code>	Merge Sort	$O(n \log n)$	Sort by author name, asc or desc

Book Class

Blueprint for one book. Bundles title, author, genre, year, price into one object so all five fields always travel together.

```
class Book:
    def __init__(self, title, author, genre, year, price):
        self.title = title # str
        self.author = author # str
        self.genre = genre # str
        self.year = year # int
        self.price = price # float
```

`self` = this specific book object. `__init__` runs automatically on creation.

Functions in Detail

1. `display_all` — $O(n)$

- Loops with index `i` so we can print `i+1` (users expect numbering from 1, not 0).
- Checks `len == 0` first and exits early if the list is empty.

```
for i in range(len(self.books)):
    b = self.books[i]
    print(str(i+1) + '. ' + b.title + ' | ' + b.author + ...)
```

2. `add_book` — $O(1)$

- `append()` places the book at the end. No shifting of existing items needed.
- $O(1)$ — same speed for 1 book or 10,000.

```
self.books.append(book)
```

3. `delete_book` — $O(n)$

- `.lower()` on both sides = case-insensitive match. 'hobbit' finds 'The Hobbit'.
- return immediately after deletion — stops loop so no second match is deleted.

```
for book in self.books:
```

```

        if book.title.lower() == title.lower():
            self.books.remove(book)
            return

```

4. search_title_author — O(n)

- Uses `in` (substring) not `==` (exact). 'Tolk' matches 'J.R.R. Tolkien'.
- No return after match — continues looping so all matches are printed.
- found flag = False before loop. Set True on any match. After loop, if still False → print 'not found'.

```

if query.lower() in book.title.lower() or query.lower() in book.author.lower():
    print(book.title + ' by ' + book.author)
    found = True

```

5. search_genre — O(n)

- Same loop and found-flag pattern as #4.
- Uses `==` not `in` — genre is a fixed category, you search for exactly 'Fantasy'.

```

if book.genre.lower() == genre.lower():

```

6. sort_by_year — Merge Sort O(n log n)

- Asks user: a = ascending (oldest first), b = descending (newest first).
- Merge Sort always produces ascending. For descending, `reverse_list()` is called after.
- Why Merge Sort: 1,000 books = ~10,000 comparisons vs Bubble Sort's 1,000,000.
- Stable — books with the same year keep their original relative order.

merge_sort_year — splits recursively until 1 item (base case), then merges.

```

def merge_sort_year(self, arr):
    if len(arr) <= 1: return arr          # base case
    mid = len(arr) // 2
    left = self.merge_sort_year(arr[:mid])
    right = self.merge_sort_year(arr[mid:])
    return self.merge_year(left, right)

```

merge_year — two pointers i and j walk both sorted halves, taking the smaller year each step.

```

while i < len(left) and j < len(right):
    if left[i].year <= right[j].year:
        result.append(left[i]); i = i + 1
    else:
        result.append(right[j]); j = j + 1
# attach any remaining items from either side

```

Merge Sort trace — 4 books:

Step	Action	Result
1	Split [2003,1937,1965,1949]	[2003,1937] and [1965,1949]
2	Split each half again	[2003],[1937] and [1965],[1949]
3	Base case — single items	All single items already sorted
4	Merge [2003],[1937]	[1937, 2003]
5	Merge [1965],[1949]	[1949, 1965]
6	Merge [1937,2003],[1949,1965]	[1937, 1949, 1965, 2003] DONE

7. search_by_year — Binary Search O(log n)

- Requires list sorted ascending by year (option 6, choose a) first.

- Jumps to middle, checks year, eliminates half the list each step.
- 1,000 books = up to 10 checks vs linear scan's 1,000.

```
low = 0; high = len(self.books) - 1
while low <= high:
    mid = (low + high) // 2
    if self.books[mid].year == year: # found
        return
    elif year < self.books[mid].year:
        high = mid - 1 # search left half
    else:
        low = mid + 1 # search right half
```

Trace — finding 1965 in 8 books:

Step	low	high	mid	books[mid].year	Decision
1	0	7	3	1988	1965 < 1988 → high = 2
2	0	2	1	1949	1965 > 1949 → low = 2
3	2	2	2	1965	Match — FOUND in 3 steps

8. lost_book_fine — O(n)

- Linear scan to find book by title (same pattern as delete).
- Fine = book.price + (days x 0.50). round(fine, 2) gives 2 decimal places.
- int(input()) converts typed string to number for arithmetic.

```
days = int(input('Days overdue: '))
fine = book.price + (days * 0.50)
print('Fine: $' + str(round(fine, 2)))
```

9. sort_by_author — Merge Sort O(n log n)

- Identical structure to sort_by_year. Asks a (A to Z) or b (Z to A).
- merge_author compares author.lower() instead of year.
- Python compares strings alphabetically — 'dan' < 'george' is True — so the same <= logic works.

```
if left[i].author.lower() <= right[j].author.lower():
    result.append(left[i]); i = i + 1
else:
    result.append(right[j]); j = j + 1
```

reverse_list — O(n) (helper)

- Used by sort_by_year and sort_by_author to flip ascending into descending.
- Reads original list backwards using index i starting at last item.

```
i = len(arr) - 1
while i >= 0:
    result.append(arr[i])
    i = i - 1
```

Menu and Main

print_menu() — standalone function, prints 10 options. Called once per loop so the lines are written only once.

main() — creates Library, loads 8 books, runs while True loop until user enters 0.

- while True = repeat forever. break on choice 0 exits the loop.

- `input()` always returns a string. Choices compare to '1' not 1.
- `int(input())` and `float(input())` convert strings to numbers when needed.

Quick Reference

Term	Meaning
$O(1)$	Constant — same speed regardless of list size
$O(n)$	Linear — scales with number of books
$O(n \log n)$	Log-linear — Merge Sort, much faster than $O(n^2)$
$O(\log n)$	Logarithmic — Binary Search, halves range each step
<code>self</code>	Refers to the current object inside a class
<code>__init__</code>	Constructor, runs automatically when object is created
Base case	Stopping condition for recursion (list of 1 item in Merge Sort)
Stable sort	Equal keys keep original relative order — Merge Sort is stable
Substring match	<code>in</code> checks if query appears anywhere in string, not exact match
<code>append()</code>	Adds item to end of list — $O(1)$
<code>remove()</code>	Removes first matching item from list — $O(n)$